Operating Systems

Task 1 & Task 2

Name: Pranay Reddy

Roll No.: 2520080061

TASK 1

Pranay Reddy | Roll No.: 2520080061

```c
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

#include <stdlib.h>


int main() {
    pid_t pid;


    printf("Before fork():\n");

    printf("Process PID: %d\n", getpid());
    printf("Parent PID: %d\n\n", getppid());


    pid = fork();


    if (pid < 0) {
        perror("fork failed");
        return 1;
    }


    if (pid == 0) {
        printf("CHILD PROCESS\n");

        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("State: Running\n");


        sleep(5);
```

```
        printf("State: Running\n");


        sleep(5);


        printf("\nChild process after sleep\n");

        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("State: Running\n");

        printf("Child process terminating...\n");
        exit(0);
    }

    else {
        printf("PARENT PROCESS\n");

        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("Child PID: %d\n", pid);
        printf("State: Running\n");


        printf("\nParent waiting for child...\n");
        wait(NULL);

        printf("Child process completed.\n");

        printf("Parent process terminating...\n");
    }


    return 0;
}
```

```
pranay@LAPTOP-OP7R42IK:~$ gcc task3.c -o task3
pranay@LAPTOP-OP7R42IK:~$ ./task3
Before fork():
Process PID: 543
Parent PID: 342

PARENT PROCESS
PID : 543
PPID: 342
Child PID: 544
State: Running

CHILD PROCESS
Parent waiting for child...
PID : 544
PPID: 543
State: Running

Child process after sleep
PID : 544
PPID: 543
State: Running
Child process terminating...
Child process completed.
Parent process terminating...
```

TASK 2

Pranay Reddy | Roll No.: 2520080061

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <time.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process Started\n");
        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());

        printf("\nState 1: WAITING/SLEEPING\n");
        sleep(10);

        printf("\nState 2: RUNNING\n");

        time_t start = time(NULL);

        while (time(NULL) - start < 15) {
            volatile unsigned long long x = 0;
            for (unsigned long long i = 0; i < 10000000; i++) {
                x += i;
            }
        }

        printf("\nState 3: TERMINATING\n");
        exit(0);
    }
```

```
    }

    else {
        printf("Parent Process\n");
        printf("PID    : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("\nParent is waiting for 35 seconds.\n");
        sleep(35);

        printf("\nParent calling wait().\n");
        wait(NULL);

        printf("Child process collected.\n");
        printf("Parent terminating.\n");
    }

    return 0;
}
```

```
Parent Process
PID   : 490
Child PID : 491

Parent is waiting for 35 seconds.
Child Process Started
PID : 491
PPID: 490

State 1: WAITING/SLEEPING

State 2: RUNNING

State 3: TERMINATING

Parent calling wait().
Child process collected.
Parent terminating.
pranay@LAPTOP-OP7R42IK:~$ nano task4.c
pranay@LAPTOP-OP7R42IK:~$
```